



石河子大学信息科学与技术学院

〈数据挖掘〉课程实验报告

实验名称：	实验 4 频繁项集挖掘算法实验
学生姓名：	卢瑶
学 号：	20211018341
学 院：	信息科学与技术学院
专业年级：	计科 2021 级 5 班
指导教师：	刘雅辉
完成日期：	2024. 4. 12

实验 4 频繁项集挖掘算法实验

一、实验目的

- 1、掌握 Apriori、FP-growth 算法的原理。
- 2、掌握 Apriori、FP-growth 算法的实现方法。

二、实验原理

1、Apriori 算法是经典的挖掘频繁项集和关联规则的数据挖掘算法。Apriori 算法使用一种称为逐层搜索的迭代方法，其中 k 项集用于探索 $(k+1)$ 项集。为了提高频繁项集逐层产生的效率，Apriori 算法使用频繁项集的先验性质来压缩搜索空间。先验性质为：

- (1) 频繁项集的所有非空子集也一定是频繁的。
- (2) 非频繁项集的所有超集一定也是非频繁项集。

2、FP-growth 算法核心思想是使用一种称为频繁模式树 (Frequent Pattern Tree, 简称 FP 树) 的数据结构来压缩数据，同时保留项集关联信息。FP-Growth 算法的执行过程主要包括两个步骤：

(1) 构建 FP 树。首先，对数据集进行两次扫描，第一次扫描用于统计所有项的频率，第二次扫描则是将不满足最小支持度要求的项过滤掉，并将剩余的项按照其在数据集中出现的顺序和频率插入到 FP 树中。

(2) 从 FP 树中挖掘频繁项集。算法会从 FP 树中获得条件模式基，并构建条件 FP 树，然后重复这一过程直到 FP 树只包含一个元素。

三、实验环境

使用 Weka 或 Python 实现 Apriori 算法和 FP-growth 算法。

四、实验数据

basket_I.txt

五、实验内容

本次实验，利用 Apriori 算法和 FP-growth 算法实现购物篮分析，最小支持度阈值设置为 0.2，最小置信度阈值设置为 0.7。

使用 Python 实现时，要求使用 [efficient-apriori](#) 库实现 Apriori 算法、使用 [mlxtend](#) 库实现 FP-growth 算法，输出结果为所有满足条件的频繁项集以及关联规则。

六、实验步骤

首先，我们分别构建 Apriori、FP-growth 算法 python 代码如下所示：

6.1 Apriori 算法代码

```
1. from efficient_apriori import apriori
2. import pandas as pd
3.
4. # 将数据集转换为列表形式，每个交易作为列表中的一个元素
5. data = [
6.     ("方便面", "蛋类食品", "肠"),
7.     ("蛋类食品", "辣条"),
```

```

8.      ("蛋类食品", "面包"),
9.      ("方便面", "蛋类食品", "辣条"),
10.     ("方便面", "面包"),
11.     ("蛋类食品", "面包"),
12.     ("方便面", "面包"),
13.     ("方便面", "蛋类食品", "面包", "肠"),
14.     ("方便面", "蛋类食品", "面包"),
15.     ("牛奶", "面包", "麦片"),
16.     ("牛奶", "面包", "糖", "鸡蛋"),
17.     ("牛奶", "面包", "黄油"),
18.     ("糖", "鸡蛋"),
19.]
20.
21.# 应用Apriori 算法
22.itemsets, rules = apriori(data, min_support=0.2, min_confidence=0.7)
23.
24.print("频繁项集: ")
25.for itemset in itemsets:
26.    print(itemset, itemsets[itemset])
27.
28.print("\n 关联规则: ")
29.for rule in rules:
30.    print(rule) # 打印出推导出的关联规则

```

6.2 FP-growth 算法代码

```

1. from mlxtend.frequent_patterns import fpgrowth
2. from mlxtend.preprocessing import TransactionEncoder
3. import pandas as pd
4.
5. # Suppose `dataset` is your list of transactions
6. dataset = [
7.     ("方便面", "蛋类食品", "肠"),
8.     ("蛋类食品", "辣条"),
9.     ("蛋类食品", "面包"),
10.    ("方便面", "蛋类食品", "辣条"),
11.    ("方便面", "面包"),
12.    ("蛋类食品", "面包"),
13.    ("方便面", "面包"),
14.    ("方便面", "蛋类食品", "面包", "肠"),
15.    ("方便面", "蛋类食品", "面包"),
16.    ("牛奶", "面包", "麦片"),
17.    ("牛奶", "面包", "糖", "鸡蛋"),
18.    ("牛奶", "面包", "黄油"),
19.    ("糖", "鸡蛋"),

```

```

20.]
21.
22.# Convert the dataset into a format suitable for the fitting the model
23.te = TransactionEncoder()
24.te_ary = te.fit(dataset).transform(dataset)
25.df = pd.DataFrame(te_ary, columns=te.columns_)
26.
27.# Apply the FP-growth algorithm
28.frequent_itemsets = fpgrowth(df, min_support=0.2, use_colnames=True)
29.
30.# You can also add a column that tells you how many items are in each itemset
31.frequent_itemsets['length'] = frequent_itemsets['itemsets'].apply(lambda x: len(x))
32.
33.print(frequent_itemsets)
34.
35.# If you also want to find and print the rules
36.from mlxtend.frequent_patterns import association_rules
37.
38.rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)
39.
40.print("-----关联规则👉-----")
41.for index, rule in rules.iterrows():
42.    print(rule)
43.# print(rules)

```

构建好以上代码框架之后，接下来的实验结果及分析如第七节所示。

七、实验结果及分析

7.1 Apriori 算法实验结果分析

```

/opt/anaconda3/envs/DeepLearning/bin/python /Users/luyao/ProgramFiles/DeepLearning/diffusion/DenoisingDiffusionProbabilityModel-ddpm--main/test.py
频繁项集:
1 {('方便面',): 6, ('蛋类食品',): 7, ('面包',): 9, ('牛奶',): 3}
2 {('方便面', '蛋类食品'): 4, ('方便面', '面包'): 4, ('牛奶', '面包'): 3, ('蛋类食品', '面包'): 4}

关联规则:
{牛奶} -> {面包} (conf: 1.000, supp: 0.231, lift: 1.444, conv: 307692307.692)

Process finished with exit code 0

```

频繁项集:

1. 第一个频繁项集是{'面包'}，出现了9次，占据了整个数据集的一定比例。
2. 第二个频繁项集是{'蛋类食品', '面包'}，出现了4次，展示了'蛋类食品'和'面包'一起出现的情况。
3. 第三个频繁项集同样是由'蛋类食品'和'面包'组成，也出现了4次。

关联规则:

- 展示的规则是{'牛奶'} -> {'面包'}。这表明在购物篮数据中,买了'牛奶'的顾客也很有可能购买'面包'。
- 置信度(confidence)为 1.00,这意味着在包含'牛奶'的所有交易中,'面包'也总是出现,这显示了一个很强的关联。
- 支持度(support)为 0.231,意味着'牛奶'和'面包'这一组合在所有交易中出现的比例约为 23.1%。
- 提升度(lift)为 1.444,这个值大于 1,表示'牛奶'和'面包'的组合比随机出现的概率要高,这是一个积极的提升度值,意味着'牛奶'的出现对'面包'的出现有正向的提升作用。

结论: 基于上述结果,可以认为'牛奶'和'面包'是强关联的产品组合,如果商家想要提升销售,可以考虑将它们放置在相近的位置,或者一起进行促销活动。

7.2 FP-growth 算法实验结果分析

```
/opt/anaconda3/envs/DeepLearning/bin/python /l
      support      itemsets  length
0  0.538462      (蛋类食品)      1
1  0.461538      (方便面)      1
2  0.692308      (面包)      1
3  0.230769      (牛奶)      1
4  0.307692  (面包, 蛋类食品)      2
5  0.307692  (方便面, 蛋类食品)      2
6  0.307692  (方便面, 面包)      2
7  0.230769  (面包, 牛奶)      2
-----关联规则 🍌 -----
antecedents              (牛奶)
consequents              (面包)
antecedent support      0.230769
consequent support      0.692308
support                  0.230769
confidence                1.0
lift                      1.444444
leverage                  0.071006
conviction                inf
zhangs_metric            0.4
Name: 0, dtype: object
```

频繁项集:

1. 频繁项集 ('蛋类食品') 的支持度为 0.538462,说明在所有的交易中,有约 53.85%包含了'蛋类食品'。

2. 频繁项集 ('方便面') 的支持度为 0.461538, 大约 46.15% 的交易中包含了 '方便面'。
3. 频繁项集 ('面包') 的支持度最高, 为 0.692308, 意味着 '面包' 出现在大约 69.23% 的交易中。
4. 频繁项集 ('牛奶') 的支持度为 0.230769, 大约在 23.08% 的交易中包含了 '牛奶'。
5. 两个项的频繁项集 ('面包', '蛋类食品') 的支持度为 0.307692, 约占 30.77%。
6. 同样, ('方便面', '蛋类食品') 和 ('方便面', '面包') 的支持度也是 0.307692。
7. ('面包', '牛奶') 的支持度为 0.230769, 和单个 '牛奶' 的支持度相同, 这表明每当 '牛奶' 被购买时, '面包' 也总被一起购买。

关联规则:

- 提出了一个规则, 其前项是 {'牛奶'}, 后项是 {'面包'}。
- 这个规则的支持度为 0.230769, 意味着 '牛奶' 和 '面包' 一起出现的交易约占有所有交易的 23.08%。
- 置信度为 1.0, 这表示所有包含 '牛奶' 的交易中, '面包' 总是同时存在, 这是一个非常强的关联。
- 提升度为 1.444444, 意味着 '牛奶' 和 '面包' 一起出现的概率是它们独立出现概率的 1.444 倍, 这显示出一个正相关。
- 杠杆率为 0.071006, 说明 '牛奶' 和 '面包' 一起出现比独立出现的概率多出 7.1%。
- Conviction 值为无穷大 (inf), 这通常表示后项在没有前项的情况下几乎不会发生, 即 '面包' 没有 '牛奶' 就几乎不会被购买, 但由于实验数据较少, 这个指标可能会出现极端值。
- Zhang's Metric 为 0.4, 表示规则的强度为中等程度。

结论: 该实验的结果提供了清晰的指示: '牛奶' 和 '面包' 是一对非常强的关联商品, 这对于零售商来说是有价值的信息。他们可以把这两种商品放在一起或进行捆绑销售, 以增加销售额。同时, '蛋类食品' 和 '方便面' 也是相对常一起购买的商品, 可以考虑一起促销。这些数据可以帮助商家制定更有效的销售策略和库存管理。

八、本次实验的主要收获和体会, Apriori 算法和 FP-growth 算法的应用场景有哪些。

8.1 主要收获和体会

通过本次实验, 我深入了解并掌握了两种主要的频繁项集挖掘算法: Apriori 和 FP-growth。这两种算法在理论和实践中均显示出其重要性和有效性。在实验过程中, 我体会到了以下几点:

- ✧ **理论与实践的结合:** 通过编写并执行 Python 代码实现这两种算法, 我能够将理论知识应用于实际问题解决中, 加深了对算法原理的理解。
- ✧ **算法效率的比较:** Apriori 算法虽然实现简单, 但在大数据集上可能效率较低, 因为它需要多次扫描数据库生成频繁项集。相比之下, FP-growth 算法通过构建 FP 树, 减少了数据扫描次数, 显示出更高的效率。
- ✧ **数据预处理的重要性:** 实验中的数据预处理, 如事务编码, 对于算法的成功实施至关重要。正确的数据格式化是挖掘频繁项集的前提。
- ✧ **参数设置的影响:** 最小支持度和置信度阈值的设定直接影响到频繁项集和关联规则的生成。这要求我们根据具体情况灵活调整这些参数以获得最佳结果。

8.2 Apriori 算法和 FP-growth 算法的应用场景

- ✧ **市场篮分析:** 这是最直接的应用场景, 通过分析顾客的购物篮, 可以发现商品之间的关

联规则，帮助商家进行有效的货架管理和交叉销售策略。

- ✧ **推荐系统:** 在电影推荐、书籍推荐等领域，通过挖掘用户的历史行为数据来发现物品之间的频繁模式，进而推荐相关联的物品。
- ✧ **生物信息学:** 在生物信息学中，可以应用这些算法来分析基因表达数据，找出经常一起表达的基因集合，这对疾病的研究和治疗具有重要意义。
- ✧ **网络安全:** 在网络交易或使用中，可以通过分析用户行为模式，识别出异常或欺诈行为，提高安全防护能力。
- ✧ **库存管理:** 在库存管理中，通过分析销售数据，可以优化库存的配置和补货计划，减少库存成本，提高经营效率。